Numerical Implementation of MPS and DMRG

Kimi 2.5 and Claude 4.6

Based on: U. Schollwöck, The density-matrix renormalization group in the age of matrix product states, Annals of Physics 326 (2011) 96-192

(Dated: April 4, 2026)

I. INTRODUCTION

This document provides implementation-ready specifications for Matrix Product State (MPS) algorithms and Density Matrix Renormalization Group (DMRG), based on U. Schollwöck's review article [1].

Critical Prerequisites:

1. **Read Section II first** before implementing any code. Index ordering bugs are the most common source of silent errors.
2. All implementations must use **complex128** (double precision) dtype, even for real Hamiltonians.
3. The effective Hamiltonian must be implemented **matrix-free** (Section 4.2). Explicit construction causes memory overflow for $D > 50$.
4. Follow the phased verification protocol (Section XI) strictly. Do not skip Phase 0.

Notation Summary:

- L: Chain length (number of sites)
- d: Physical dimension per site (e.g., $d = 2$ for spin-1/2)
- D: MPS bond dimension (controls accuracy)
- D_W: MPO bond dimension (e.g., $D_W = 5$ for Heisenberg)
- $A^{(i)}_{a_{i-1},\sigma_i,a_i}$: Left-canonical MPS tensor at site i
- $W^{(i)}_{b_{i-1},b_i,\sigma,\sigma'}$: MPO tensor (Hamiltonian)
- $L^{(i)}_{b_i}(a_{i-1}, a'_{i-1})$: Left environment (shape (D_W, D_{i-1}, D_{i-1}))
- $R^{(i)}_{b_i}(a_i, a'_i)$: Right environment (shape (D_W, D_i, D_i))

Index Convention (Zero-Based): All indices run from 0 to $N-1$ (Python/C convention). Sites are numbered $i \in \{1, 2, \ldots, L\}$ in mathematical notation but stored as `sites[0]` to `sites[L-1]` in arrays.

II. INDEX CONVENTIONS AND STORAGE ORDER (CRITICAL)

This section defines the exact mapping between mathematical notation and array storage. Violating these conventions will cause silent bugs with incorrect results.

A. MPS Tensor Storage

Mathematical notation: $A^{(i)}_{a_{i-1},\sigma_i,a_i}$ where:

- $a_{i-1} \in [0, D_{i-1} - 1]$: left bond index
- $\sigma_i \in [0, d-1]$: physical index
- $a_i \in [0, D_i - 1]$: right bond index

NumPy array storage: ‘A‘ has shape (D_{i-1}, d, D_i) with C-order (row-major) indexing:

$$\boxed{A[j,k,l] = A^{(i)}_{a_{i-1}=j,\sigma_i=k,a_i=l}} \tag{1}$$

Verification code:

```
A = np.random.rand(D_left, d, D_right)
assert A.shape == (D_left, d, D_right)
# Access element: a_left=2, sigma=1, a_right=3
element = A[2, 1, 3]  # Correct
# element = A[1, 2, 3]  # WRONG - swapped indices
```

B. MPO Tensor Storage

Mathematical notation: $W^{(i)}_{b_{i-1},b_i,\sigma,\sigma'}$ represents operator $\sum_{\sigma,\sigma'} W_{b_{i-1},b_i,\sigma,\sigma'}|\sigma'\rangle\langle\sigma|$
Physical index convention:

- σ: input state (bra side, contracts with ket MPS)
- σ': output state (ket side, contracts with bra MPS)

NumPy array storage: ‘W‘ has shape (D_W, D_W, d, d):

$$\boxed{W[b_L, b_R, s_{in}, s_{out}] = W^{(i)}_{b_{i-1}=b_L,b_i=b_R,\sigma=s_{in},\sigma'=s_{out}}} \tag{2}$$

Action on state: $(\hat{W}|\psi\rangle)_{\sigma'} = \sum_\sigma W_{b_L,b_R,\sigma,\sigma'}\psi_\sigma$
Verification for Heisenberg S^+ operator:

```
Sp = np.array([[0, 1], [0, 0]])  # Raises spin
# In MPO: W[row, col, :, :] = Sp means:
# W[row, col, 0, 1] = 1  (input |down>, output |up>)
# W[row, col, 1, 0] = 0  (input |up>, output |down>)
assert W[1, 0, 0, 1] == 1.0  # S^+ at position [1,0]
assert W[1, 0, 1, 0] == 0.0
```

C. Environment Tensor Storage

Left environment: $L^{(i)}_{b_i}(a_{i-1}, a'_{i-1})$ is a $D_{i-1} \times D_{i-1}$ matrix for each MPO index b_i.
NumPy storage: ‘L‘ has shape (D_W, D_{i-1}, D_{i-1}):

$$\boxed{L[b, a, a_prime] = L^{(i)}_{b_i=b}(a_{i-1} = a, a'_{i-1} = a_prime)} \tag{3}$$

Right environment: ‘R‘ has shape (D_W, D_i, D_i):

$$\boxed{R[b, a, a_prime] = R^{(i)}_{b_i=b}(a_i = a, a'_i = a_prime)} \tag{4}$$

Initialization:

```
# Left boundary (site 1)
L_init = np.zeros((D_W, 1, 1), dtype=complex)
L_init[4, 0, 0] = 1.0  # b_left = 4 for Heisenberg (D_W=5)

# Right boundary (site L)
R_init = np.zeros((D_W, 1, 1), dtype=complex)
R_init[0, 0, 0] = 1.0  # b_right = 0 for Heisenberg (D_W=5)
```

D. Complex Conjugation Convention

Left environment update (Eq. 2):

$$L^{(i)}_{b_i}(a_{i-1}, a'_{i-1}) = \sum_{a_{i-2}, a'_{i-2}} \sum_{\sigma,\sigma'} \sum_{b_{i-1}} A^{(i-1)*}_{a'_{i-2},\sigma',a'_{i-1}} \, L^{(i-1)}_{b_{i-1}}(a_{i-2}, a'_{i-2}) \, W^{(i-1)}_{b_{i-1},b_i,\sigma,\sigma'} \, A^{(i-1)}_{a_{i-2},\sigma,a_{i-1}} \tag{5}$$

Conjugation rule: $A^{(i)*}$ means **element-wise complex conjugate only, NO transpose**:

$$\boxed{A^{(i)*}_{a,\sigma,a'} \equiv \overline{A^{(i)}_{a,\sigma,a'}} \quad \text{(NumPy: } \texttt{A.conj()}\text{)}} \tag{6}$$

WRONG: $A^{(i)*} \neq A^{(i)\dagger}$ (conjugate transpose). The dagger notation $A^\dagger$ would swap indices, but we only conjugate values.

Right environment update (Eq. 3):

$$R^{(i)}_{b_i}(a_i, a'_i) = \sum_{a_{i+1}, a'_{i+1}} \sum_{\sigma,\sigma'} \sum_{b_{i+1}} B^{(i+1)}_{a_i,\sigma,a_{i+1}} \, R^{(i+1)}_{b_{i+1}}(a_{i+1}, a'_{i+1}) \, W^{(i+1)}_{b_i,b_{i+1},\sigma,\sigma'} \, B^{(i+1)*}_{a'_i,\sigma',a'_{i+1}} \tag{7}$$

Same rule: $B^{(i+1)*}$ is element-wise conjugate, no transpose.

Python implementation:

```
# CORRECT
A_conj = A.conj()  # Shape unchanged: (D_left, d, D_right)

# WRONG - do not use these
A_dagger = A.conj().T  # Wrong! Changes shape
A_herm = np.conjugate(A).transpose(2, 1, 0)  # Wrong! Swaps indices
```

E. Einsum Index Mapping (Definitive)

Left environment update with explicit index correspondence:

Mathematical	Array name
b_{i-1}	`b` (old MPO bond)
b_i	`B` (new MPO bond)
a_{i-2}	`x` (old left MPS bond, ket)
a'_{i-2}	`y` (old left MPS bond, bra)
a_{i-1}	`a` (new left MPS bond, ket)
a'_{i-1}	`A` (new left MPS bond, bra)
σ	`s` (physical, ket)
σ'	`t` (physical, bra)

Einsum formula:

```
# L_new[B, a, A] = sum over b, x, y, s, t of:
#   L_old[b, x, y] * A[x, s, a] * W[b, B, s, t] * A_conj[y, t, A]
L_new = np.einsum('bxy,xsa,bBst,ytA->BaA', L_old, A, W, A.conj())
```

Right environment update:

Mathematical	Array name
b_{i+1}	`B` (old MPO bond)
b_i	`b` (new MPO bond)
a_i	`a` (old right MPS bond, ket)
a'_i	`A` (old right MPS bond, bra)
a_{i+1}	`x` (new right MPS bond, ket)
a'_{i+1}	`y` (new right MPS bond, bra)
σ	`s` (physical, ket)
σ'	`t` (physical, bra)

Einsum formula (CORRECTED):

```
# R_new[b, a, A] = sum over B, x, y, s, t of:
#   B[a, s, x] * R_old[B, x, y] * W[b, B, s, t] * B_conj[A, t, y]
# Note: W[b, B, s, t] = W_{new, old, sigma, sigma'} where b=new, B=old
R_new = np.einsum('asx,Bxy,bBst,Aty->baA', B, R_old, W, B.conj())
# Verify Hermiticity after computation
assert all(np.allclose(R_new[b], R_new[b].conj().T) for b in range(D_W))
```

F. Reshape Convention for SVD/QR

When converting 3-index tensor to matrix for decomposition:
Left-canonicalization (QR): Merge left bond and physical index as rows:

$$\boxed{M_{\text{matrix}}[r, c] = M_{\text{tensor}}[a_{\text{left}}, \sigma, a_{\text{right}}] \quad \text{where } r = a_{\text{left}} \cdot d + \sigma} \tag{8}$$

Python:

```
# Tensor shape: (D_left, d, D_right)
# Matrix shape: (D_left * d, D_right)
M_matrix = M.reshape(D_left * d, D_right, order='C')

# Verify mapping
assert M_matrix[a_left * d + sigma, a_right] == M_tensor[a_left, sigma, a_right]
```

Right-canonicalization (RQ or reverse QR): Merge physical and right bond as columns:

$$\boxed{M_{\text{matrix}}[r, c] = M_{\text{tensor}}[a_{\text{left}}, \sigma, a_{\text{right}}] \quad \text{where } c = \sigma \cdot D_{\text{right}} + a_{\text{right}}} \tag{9}$$

Python:

```
# Tensor shape: (D_left, d, D_right)
# Matrix shape: (D_left, d * D_right)
M_matrix = M.reshape(D_left, d * D_right, order='C')

# Verify mapping
assert M_matrix[a_left, sigma * D_right + a_right] == M_tensor[a_left, sigma, a_right]
```

G. Canonical Form Verification

Left-canonical tensor $A^{(i)}$ satisfies:

$$\sum_{\sigma=0}^{d-1} \sum_{a_{i-1}=0}^{D_{i-1}-1} \overline{A^{(i)}_{a_{i-1},\sigma,a_i}} \cdot A^{(i)}_{a_{i-1},\sigma,a'_i} = \delta_{a_i,a'_i} \tag{10}$$

Python verification:

```
# Method 1: Direct summation
identity = np.einsum('xsa,xsb->ab', A.conj(), A)
assert np.allclose(identity, np.eye(D_right), atol=1e-14)

# Method 2: Matrix form
A_matrix = A.reshape(D_left * d, D_right, order='C')
identity = A_matrix.conj().T @ A_matrix
assert np.allclose(identity, np.eye(D_right), atol=1e-14)
```

Right-canonical tensor $B^{(i)}$ satisfies:

$$\sum_{\sigma=0}^{d-1} \sum_{a_i=0}^{D_i-1} \overline{B^{(i)}_{a_{i-1},\sigma,a_i}} \cdot B^{(i)}_{a'_{i-1},\sigma,a_i} = \delta_{a_{i-1},a'_{i-1}} \tag{11}$$

Python verification:

```
# Method 1: Direct summation (use consistent index labels)
identity = np.einsum('asb,Asb->aA', B.conj(), B)
assert np.allclose(identity, np.eye(D_left), atol=1e-14)

# Method 2: Matrix form
B_matrix = B.reshape(D_left, d * D_right, order='C')
identity = B_matrix @ B_matrix.conj().T
assert np.allclose(identity, np.eye(D_left), atol=1e-14)
```

H. Common Index Bugs and Detection

TABLE I. Typical Index Errors

Bug	**Symptom**	**Detection**
Swapped MPS indices	Non-orthogonal A	$\|A^\dagger A - I\| > 10^{-10}$
Wrong MPO σ, σ' order	Wrong energy sign	Compare $L = 2$ to exact
Missing conjugate	Non-Hermitian L or R	$\|L - L^\dagger\| > 10^{-14}$
Transpose instead of conj	Wrong MPS overlap	$\|\langle\psi\|\psi\rangle\| \neq 1$
Wrong einsum indices	Random energies	Variance $\sigma^2 \sim E^2$
C/Fortran order mismatch	Scrambled state	Correlation functions nonsensical

Mandatory checks after each operation:

```
# After left-canonicalization
assert np.allclose(np.einsum('xsa,xsb->ab', A.conj(), A), np.eye(D_right))

# After environment update
for b in range(D_W):
assert np.allclose(L[b], L[b].conj().T, atol=1e-13), f"L[{b}] not Hermitian"

# After full sweep
norm = np.einsum('xsa,xsa->', psi.conj(), psi)
assert np.abs(norm - 1.0) < 1e-12, f"MPS not normalized: {norm}"
```

I. Summary: Quick Reference Card

```
# MPS tensor
```

```
A[a_left, sigma, a_right]  # Shape: (D_left, d, D_right)

# MPO tensor
W[b_left, b_right, sigma_in, sigma_out]  # Shape: (D_W, D_W, d, d)

# Environments
L[b, a, a_prime]  # Shape: (D_W, D_left, D_left)
R[b, a, a_prime]  # Shape: (D_W, D_right, D_right)

# Complex conjugation (NO transpose)
A_conj = A.conj()  # Same shape as A

# Left environment update
L_new = np.einsum('bxy,xsa,bBst,ytA->BaA', L_old, A, W, A.conj())

# Right environment update (CORRECTED)
# Note: B=old MPO bond, b=new MPO bond in W[b,B,s,t]
R_new = np.einsum('asx,Bxy,bBst,Aty->baA', B, R_old, W, B.conj())

# Reshape for left-canonicalization (QR)
M_matrix = M.reshape(D_left * d, D_right, order='C')

# Reshape for right-canonicalization (RQ)
M_matrix = M.reshape(D_left, d * D_right, order='C')

# Verification: left-canonical
assert np.allclose(np.einsum('xsa,xsb->ab', A.conj(), A), np.eye(D_right))

# Verification: right-canonical
assert np.allclose(np.einsum('asb,Asb->aA', B.conj(), B), np.eye(D_left))

# Verification: Hermitian environment
assert np.allclose(L[b], L[b].conj().T) for all b
```

III. DATA STRUCTURES

An MPS with open boundary conditions is stored as a list of 3-index tensors:

- $A^{(i)}_{a_{i-1},\sigma_i,a_i}$: Left-canonical tensor at site i, dimensions $(D_{i-1} \times d \times D_i)$
- $B^{(i)}_{a_{i-1},\sigma_i,a_i}$: Right-canonical tensor, dimensions $(D_{i-1} \times d \times D_i)$
- $M^{(i)}_{a_{i-1},\sigma_i,a_i}$: General (non-canonical) tensor during optimization
- Physical dimension d (e.g., $d = 2$ for spin-1/2)

Storage Convention: For site i, tensor $T^{(i)}$ has shape (D_{i-1}, d, D_i) where the middle index is physical.

Matrix Reshape Convention (Critical): When flattening $T^{(i)}_{a_{i-1},\sigma_i,a_i}$ to matrix $\tilde{M}_{r,c}$ for QR/SVD, the row index r must follow C-order (row-major) mapping:

$$r = a_{i-1} \cdot d + \sigma_i \quad \text{where } a_{i-1} \in [0, D_{i-1} - 1], \sigma_i \in [0, d-1] \tag{12}$$

Column index $c = a_i$ (right bond). This yields matrix shape $(D_{i-1}d) \times D_i$.

Python/NumPy Implementation:

```
# C-order (row-major) reshape - CRITICAL for index consistency
M_tilde = M.reshape((D_left * d, D_right), order='C')
# Verify: M_tilde[a_prev * d + sigma, a_curr] == M[a_prev, sigma, a_curr]
```

Fortran/MATLAB: Use `reshape(..., order='F')` with reversed index ordering $r = \sigma_i \cdot D_{i-1} + a_{i-1}$ (column-major), or transpose after C-order reshape.

IV. CANONICALIZATION ALGORITHMS

A. Left-Canonicalization (QR-based)

More stable than SVD for canonicalization when singular values aren't needed.

Algorithm 1 LeftCanonicalize (Numerically Stable)

Require: MPS tensors $\{M^{(1)}, \ldots, M^{(L)}\}$ with shapes (D_{i-1}, d, D_i)
Ensure: Left-canonical $\{A^{(1)}, \ldots, A^{(L)}\}$, last tensor C (scalar or vector)

$C \leftarrow 1.0$ ▷ Initial transfer matrix, 1×1
for $i = 1$ to $L - 1$ **do**
▷ Contract incoming transfer matrix C with current site
  $\tilde{M}_{(a_{i-1}\sigma_i),a_i} \leftarrow \sum_{a'_{i-1}} C_{a_{i-1},a'_{i-1}} M^{(i)}_{a'_{i-1},\sigma_i,a_i}$
▷ For $i = 1$: C is 1×1, so $\tilde{M}$ is simply $M^{(1)}$ reshaped to $(d) \times D_1$
▷ Matrix shape: $(D_{i-1}d) \times D_i$
▷ QR decomposition with column-pivoting for stability
  if $D_{i-1} \cdot d < D_i$ **then**
    Warning: Bond dimension D_i exceeds physical capacity $D_{i-1}d$.
    State is necessarily rank-deficient. Perform truncation or use random initialization.
  end if
  $Q, R \leftarrow \mathrm{QR}(\tilde{M})$ ▷ Q: $(D_{i-1}d) \times D_i$, R: $D_i \times D_i$
  $A^{(i)}_{a_{i-1},\sigma_i,a_i} \leftarrow \mathrm{reshape}(Q, (D_{i-1}, d, D_i))$
  $C \leftarrow R$ ▷ Transfer matrix to next site, shape $D_i \times D_i$
end for
▷ Final site: absorb last C and normalize
$\tilde{A}^{(L)}_{a_{L-1},\sigma_L,1} \leftarrow \sum_{a'_{L-1}} C_{a_{L-1},a'_{L-1}} M^{(L)}_{a'_{L-1},\sigma_L,1}$
$A^{(L)} \leftarrow \tilde{A}^{(L)} / \|\tilde{A}^{(L)}\|_F$ ▷ Normalize to unit norm

B. Mixed-Canonical Form at Bond k

Required for variational minimization.

Algorithm 2 BringToBond

Numerical Note: RQ decomposition (used for right-canonicalization) is less stable than QR and may not be available in all libraries (e.g., PyTorch). Alternative: Reverse the site order, apply LeftCanonicalize, then reverse back.

Require: MPS $\{M^{(i)}\}$, target bond k
Ensure: Left-canonical $A^{(1..k)}$, right-canonical $B^{(k+1..L)}$, center matrix S

Left-canonicalize sites $1 \ldots k$ using QR sweeps
Right-canonicalize sites $L \ldots k+1$ using RQ sweeps (reverse direction)
$S \leftarrow$ matrix at bond k (contains singular values if from SVD)

V. TRUNCATION AND COMPRESSION

A. SVD Compression with Tolerance

Algorithm 3 TruncateBond

Require: Matrix M (dimensions $m \times n$), max bond $D_{\max}$, tolerance ϵ, **abs tolerance** $\epsilon_{\mathbf{abs}} \sim 10^{-14}$
Ensure: Truncated matrices $U, S, V^\dagger$ with $D \leq D_{\max}$

```
U, S, V† ← SVD(M, full_matrices=False)
s_i ← diag(S) in descending order
                                  ▷ Determine effective D using relative and absolute tolerances
                                  ▷ Safety check: prevent D = 0 when all singular values vanish
if s_0 < ε_abs then
    Warning: All singular values below absolute tolerance.
    Set D ← 1 (keep single largest, even if numerically zero).
else
    D ← min(D_max, count(s_i > max(ε · s_0, ε_abs)))
    where ε_abs ~ 10^-14 (double precision) prevents D → ∞ when s_0 ≈ 0
end if
                                  ▷ Truncation error
η ← Σ_{i=D}^{r-1} s_i^2           ▷ Discarded weight
U ← U_{:,0:D-1}, S ← S_{0:D-1,0:D-1}, V† ← V†_{0:D-1,:}
return U, S, V†, η
```

Numerical Note: For $\epsilon \sim 10^{-12}$, single-precision floats may underflow. Use double precision (complex128).

VI. GROUND STATE ALGORITHM: SINGLE-SITE DMRG

A. Effective Hamiltonian Construction

The local Hamiltonian at site i is built from pre-computed environments:

$$L^{(i)}_{b_i}(a_{i-1}, a'_{i-1}) = \sum_{a_{i-2}, a'_{i-2}} \sum_{\sigma,\sigma'} \sum_{b_{i-1}} A^{(i-1)*}_{a'_{i-2},\sigma',a'_{i-1}} \, L^{(i-1)}_{b_{i-1}}(a_{i-2}, a'_{i-2}) \, W^{(i-1)}_{b_{i-1},b_i,\sigma,\sigma'} \, A^{(i-1)}_{a_{i-2},\sigma,a_{i-1}} \tag{13}$$

$$R^{(i)}_{b_i}(a_i, a'_i) = \sum_{a_{i+1}, a'_{i+1}} \sum_{\sigma,\sigma'} \sum_{b_{i+1}} B^{(i+1)}_{a_i,\sigma,a_{i+1}} \, R^{(i+1)}_{b_{i+1}}(a_{i+1}, a'_{i+1}) \, W^{(i+1)}_{b_i,b_{i+1},\sigma,\sigma'} \, B^{(i+1)*}_{a'_i,\sigma',a'_{i+1}} \tag{14}$$

where $W^{(i)}$ is the MPO tensor at site i.

See Section II for complete index conventions and storage order.

Python Implementation:

```
def update_left_env(L_old, A, W):
"""Update left environment from site i-1 to i.

Args:
L_old: shape (D_W, D_{i-2}, D_{i-2})
A: shape (D_{i-2}, d, D_{i-1})
W: shape (D_W, D_W, d, d)
Returns:
L_new: shape (D_W, D_{i-1}, D_{i-1})
"""
# Einsum: b=old MPO, B=new MPO, x=old left(ket), y=old left(bra),
#         a=new left(ket), A=new left(bra), s=phys(ket), t=phys(bra)
L_new = np.einsum('bxy,xsa,bBst,ytA->BaA', L_old, A, W, A.conj())

# Verify Hermiticity
```

```
for b in range(L_new.shape[0]):
assert np.allclose(L_new[b], L_new[b].conj().T, atol=1e-13)

return L_new

def update_right_env(R_old, B, W):
"""Update right environment from site i+1 to i.

Args:
R_old: shape (D_W, D_{i+1}, D_{i+1})
B: shape (D_i, d, D_{i+1})
W: shape (D_W, D_W, d, d)
Returns:
R_new: shape (D_W, D_i, D_i)
"""
# Einsum: b=new MPO, B=old MPO, a=old right(ket), A=old right(bra),
#         x=new right(ket), y=new right(bra), s=phys(ket), t=phys(bra)
# Note: W[b, B, s, t] = W_{new, old, sigma, sigma'}
R_new = np.einsum('asx,Bxy,bBst,Aty->baA', B, R_old, W, B.conj())

# Verify Hermiticity
for b in range(R_new.shape[0]):
assert np.allclose(R_new[b], R_new[b].conj().T, atol=1e-13)

return R_new
```

Index Dependency Verification:

- $L^{(i)}$ depends on sites $1 \dots i-1$, hence uses $W^{(i-1)}$ (not $W^{(i)}$)
- $R^{(i)}$ depends on sites $i+1 \dots L$, hence uses $W^{(i+1)}$ (not $W^{(i)}$)
- **Common bug**: Off-by-one error in MPO index. Verify: $L^{(2)}$ uses $W^{(1)}$, $R^{(L-1)}$ uses $W^{(L)}$.

Boundary Consistency: For open boundaries:

```
# Initialize left boundary (before site 1)
L_init = np.zeros((D_W, 1, 1), dtype=complex)
L_init[-1, 0, 0] = 1.0  # b_left = 4 for Heisenberg (D_W=5)
# Initialize right boundary (after site L)
R_init = np.zeros((D_W, 1, 1), dtype=complex)
R_init[0, 0, 0] = 1.0   # b_right = 0 for Heisenberg (D_W=5)
```

B. Local Optimization

CRITICAL: Matrix-Free Implementation Required

The effective Hamiltonian at site i acts on a vector space of dimension $dD_{i-1}D_i$. For typical DMRG parameters ($d = 2$, $D \sim 100 - 1000$), explicit construction as a matrix would require:

$$\text{Memory} = (dD_{i-1}D_i)^2 \times 16 \text{ bytes} \sim 10^{10}\text{–}10^{12} \text{ bytes (TB scale)} \tag{15}$$

The effective Hamiltonian must NEVER be constructed explicitly. Instead, implement only its action on vectors via tensor network contractions.

Solve the eigenvalue problem for the center tensor $M^{(i)}_{\alpha_{i-1},\alpha_i}$ reshaped as vector v of length $dD_{i-1}D_i$:

$$\mathcal{H}_{\text{eff}}\, v = Ev \tag{16}$$

where formally

$$\mathcal{H}_{\text{eff}} = \sum_{b_{i-1},b_i} L_{b_{i-1}} \otimes W^{(i)}_{b_{i-1},b_i} \otimes R_{b_i} \tag{17}$$

Implementation: Use iterative eigensolvers (Lanczos, Davidson, LOBPCG) that only require matrix-vector products. Supply the action $\mathcal{H}_{\text{eff}}v$ as a function/operator, not a matrix.

Lanczos stopping criterion: $\|\mathcal{H}_{\text{eff}}v - Ev\| < 10^{-10}$ or $N_{\text{iter}} > 20\text{-}50$.

Complexity Comparison:

- **Explicit matrix**: Memory $\mathcal{O}(D^6d^2)$, construction $\mathcal{O}(D^6d^2D_W)$
- **Matrix-free**: Memory $\mathcal{O}(D^3dD_W)$, per matvec $\mathcal{O}(D^3d^2D_W + D^3dD_W^2)$

For $D = 200$, $d = 2$, $D_W = 5$: explicit requires ∼640 GB, matrix-free requires ∼16 MB.

Algorithm 4 ApplyEffectiveHamiltonian (Matrix-Free - REQUIRED)

Require: Vector v (flattened from V_{a_{i-1},σ_i,a_i}), environments L shape (D_W, D_{i-1}, D_{i-1}), R shape (D_W, D_i, D_i), MPO W shape (D_W, D_W, d, d)

Ensure: $(\mathcal{H}_{\text{eff}}v)_{a_{i-1},\sigma'_i,a_i}$ **without ever constructing $\mathcal{H}_{\text{eff}}$ as matrix**

Reshape $v \to V_{a_{i-1},\sigma_i,a_i}$ ▷ Shape (D_{i-1}, d, D_i)
Step 1 (Left contraction):
  $X_{b_{i-1},a_{i-1},\sigma_i,a_i} = \sum_{a'_{i-1}} L_{b_{i-1},a_{i-1},a'_{i-1}} V_{a'_{i-1},\sigma_i,a_i}$
  einsum: `'bxy,xsa->bxsa'` ▷ Cost: $\mathcal{O}(D_W D^2 dD)$
Step 2 (MPO application):
  $Y_{b_i,a_{i-1},\sigma'_i,a_i} = \sum_{b_{i-1},\sigma_i} W_{b_{i-1},b_i,\sigma_i,\sigma'_i} X_{b_{i-1},a_{i-1},\sigma_i,a_i}$
  einsum: `'bBst,bxsa->Bxta'` ▷ Cost: $\mathcal{O}(D_W^2 D^2 d^2)$
Step 3 (Right contraction - CORRECTED):
  $(\mathcal{H}v)_{a_{i-1},\sigma'_i,a_i^{\text{new}}} = \sum_{b_i,a_i^{\text{old}}} R_{b_i,a_i^{\text{new}},a_i^{\text{old}}} Y_{b_i,a_{i-1},\sigma'_i,a_i^{\text{old}}}$
  einsum: `'Byz,Bxtz->xty'`
  where $R[b_i, a_{\text{new}}, a_{\text{old}}]$ and $Y[b_i, a_{i-1}, \sigma'_i, a_{\text{old}}]$
  Output: $[a_{i-1}, \sigma'_i, a_i^{\text{new}}]$ ▷ Cost: $\mathcal{O}(D_W D^2 dD)$
Flatten result to vector of length $dD_{i-1}D_i$
return Result vector **(never stored full matrix)**

Python Implementation Template (CORRECTED):

```
def apply_Heff(v, L, W, R):
"""Matrix-free Heff action. NEVER construct full matrix."""
D_left, d, D_right = v.shape
# Step 1: Left environment
# L[b, x, y] @ v[y, s, a] -> X[b, x, s, a]
X = np.einsum('bxy,ysa->bxsa', L, v)
# Step 2: MPO
# W[b, B, s, t] @ X[b, x, s, a] -> Y[B, x, t, a]
Y = np.einsum('bBst,bxsa->Bxta', W, X)
# Step 3: Right environment (CORRECTED)
# R[B, a_new, a_old] @ Y[B, x, t, a_old] -> Hv[x, t, a_new]
# Note: Contract over B (MPO bond) and a_old (shared bond index)
Hv = np.einsum('Bac,Bxtc->xta', R, Y)
return Hv.reshape(D_left * d * D_right)

# Use with scipy.sparse.linalg.eigsh (Lanczos)
from scipy.sparse.linalg import LinearOperator, eigsh

def matvec(v_flat):
v = v_flat.reshape((D_left, d, D_right))
return apply_Heff(v, L, W, R)
```

```
H_op = LinearOperator(shape=(d*D_left*D_right, d*D_left*D_right),
matvec=matvec, dtype=complex)
E, psi = eigsh(H_op, k=1, which='SA', tol=1e-10)
```

Verification of Matrix-Free Implementation:

- For small test cases ($D \leq 10$), construct $\mathcal{H}_{\text{eff}}$ explicitly and verify:

$$\|H_{\text{explicit}} \cdot v - \texttt{apply_Heff}(v)\|_{\max} < 10^{-14} \tag{18}$$

- Monitor memory usage: should scale as $\mathcal{O}(D^3)$, not $\mathcal{O}(D^6)$
- Profile computation time: matrix-vector product should take ∼ms, not seconds

Total cost per Lanczos iteration: $\mathcal{O}(D^3 d^2 D_W + D^3 d D_W^2)$. For sparse MPOs (Heisenberg), exploit zero blocks to reduce $D_W^2 \to D_W^{\text{eff}} \sim 3$.

C. Environment Initialization

Before sweeping, initialize boundary environments consistent with the MPO structure:

$$L_{b_0}^{(1)}(a_0, a_0') = \delta_{b_0, b_{\text{left}}} \delta_{a_0, a_0'} \qquad \text{(Left boundary: } 1 \times 1 \text{ matrices)} \tag{19}$$

$$R_{b_L}^{(L)}(a_L, a_L') = \delta_{b_L, b_{\text{right}}} \delta_{a_L, a_L'} \qquad \text{(Right boundary: } 1 \times 1 \text{ matrices)} \tag{20}$$

where b_{left} and b_{right} are the MPO indices selecting the Hamiltonian scalar (e.g., for the Heisenberg MPO in Sec. VIII, $b_{\text{left}} = 4$ (last row), $b_{\text{right}} = 0$ (first column)).

D. Complex Arithmetic Handling

Even for real Hamiltonians (e.g., Heisenberg), maintain `complex128` dtype throughout:

- **Left environment:** Uses complex conjugate only (no transpose): $L^{(i)} \leftarrow A^{(i-1)*} \cdots$ (Python: `A.conj()`, **not** `A.T.conj()`).
- **Right environment:** Uses element-wise complex conjugate only (NO transpose): $R^{(i)} \leftarrow \cdots B^{(i+1)*}$ (Python: `B.conj()`, **not** `B.conj().T`).
- **Verification:** The density matrix $\rho = MM^\dagger$ must be Hermitian. Check $\|\rho - \rho^\dagger\|_{\max} < 10^{-14}$ to detect index ordering bugs.

E. Datatype and Library Requirements

Floating Point Precision: All tensors must use `complex128` (double precision) even for real Hamiltonians. Single precision (`float32`) leads to underflow in $\epsilon \sim 10^{-12}$ truncations.

QR Decomposition: Standard `numpy.linalg.qr` lacks column-pivoting. Use:

```
from scipy.linalg import qr
Q, R, P = qr(M, pivoting=True, mode='economic')
# Reorder Q according to permutation P if necessary
```

Ensure diagonal elements of R have non-negative real parts (multiply column j of Q by $\text{sign}(R_{jj})$ if needed).

SVD Truncation: Explicitly sort singular values descending:

```
U, s, Vh = svd(M, full_matrices=False)
# Ensure descending (NumPy default) - reverse if using different library
idx = np.argsort(s)[::-1]
s, U, Vh = s[idx], U[:, idx], Vh[idx, :]
```

F. Sweep Algorithm

Algorithm 5 SingleSiteDMRG (with Gauge Control)

Require: Hamiltonian MPO $\{W^{(i)}\}$, initial MPS guess $\{M^{(i)}\}$, max bond $D_{\max}$, tolerance ϵ
Ensure: Ground state energy E_0 and optimized MPS in mixed-canonical form

Right-canonicalize initial MPS using RQ sweeps (or reverse QR)
Initialize all $R^{(i)}$ environments via backward contraction from site L
$L^{(1)} \leftarrow$ left boundary tensor (see initialization above)
while not converged **do**
  Right sweep ($i = 1$ to $L - 1$):
  for $i = 1$ to $L - 1$ **do**
    CRITICAL: Use matrix-free $\mathcal{H}_{\text{eff}}$ via Algorithm 4
    ▷ NEVER construct $\mathcal{H}_{\text{eff}}$ as $(dD_{i-1}D_i) \times (dD_{i-1}D_i)$ matrix
    ▷ Dimensions can be 10^4-10^6, making explicit matrix $\sim$ TB
    Build environments $L^{(i)}$, $R^{(i)}$ from previous updates
    Define matvec: $v \mapsto \mathcal{H}_{\text{eff}} v$ using tensor contractions (Alg. 4)
    Solve $\mathcal{H}_{\text{eff}} v = Ev$ using Lanczos/Davidson (matrix-free)
    Reshape $v \to M^{(i)}_{a_{i-1},\sigma_i,a_i}$
    $M^{(i)} \leftarrow M^{(i)}/\|M^{(i)}\|_F$ ▷ Normalize eigenvector
    Reshape $M^{(i)} \to \tilde{M}_{(a_{i-1}\sigma_i),a_i}$ (matrix $(dD_{i-1}) \times D_i$)
    $U, S, V^\dagger \leftarrow \text{SVD}(\tilde{M})$, truncate to $D_{\max}$
    $D_{\text{new}} \leftarrow \min(D_{\max}, \text{len}(S))$
    ▷ Fix gauge: make first element of each singular vector real positive
    for $j = 0$ to $D_{\text{new}} - 1$ **do**
      $\phi_j \leftarrow \arg(U_{0,j})$
      $U_{:,j} \leftarrow U_{:,j} \cdot e^{-i\phi_j}$, $V^\dagger_{j,:} \leftarrow V^\dagger_{j,:} \cdot e^{i\phi_j}$
    end for
    $A^{(i)} \leftarrow \text{reshape}(U, (D_{i-1}, d, D_{\text{new}}))$
    ▷ Absorb singular values into next site
    $M^{(i+1)}_{a_i,\sigma_{i+1},a_{i+1}} \leftarrow \sum_{a'_i} (SV^\dagger)_{a_i,a'_i} M^{(i+1)}_{a'_i,\sigma_{i+1},a_{i+1}}$
    Python: `M[i+1] = np.einsum('ij,jkl->ikl', S @ Vh, M[i+1])`
    Update $L^{(i+1)}$ using $A^{(i)}$ and $W^{(i)}$ (Eq. 2)
  end for
  Left sweep ($i = L$ to 2):
  for $i = L$ to 2 **do**
    Solve eigenproblem using matrix-free $\mathcal{H}_{\text{eff}}$ (Algorithm 4)
    Reshape $M^{(i)} \to \tilde{M}_{a_{i-1},(\sigma_i a_i)}$ (matrix $D_{i-1} \times (dD_i)$)
    $U, S, V^\dagger \leftarrow \text{SVD}(\tilde{M})$, truncate to $D_{\max}$
    $D_{\text{new}} \leftarrow \min(D_{\max}, \text{len}(S))$
    $B^{(i)} \leftarrow \text{reshape}(V^\dagger, (D_{\text{new}}, d, D_i))$
    $M^{(i-1)} \leftarrow M^{(i-1)} US$ ▷ Absorb singular values
    Update $R^{(i-1)}$ using $B^{(i)}$ and $W^{(i)}$ (Eq. 3)
  end for
  Check convergence (primary criteria):
    1. Energy: $|\Delta E| = |E_n - E_{n-1}| < \epsilon_E$ (e.g., 10^{-8} per site)
    2. Variance: $\sigma^2 = \langle\psi|H^2|\psi\rangle - E^2 < \epsilon_\sigma$ (e.g., 10^{-6})
  If both pass: converged. **Else if** sweep > max_sweeps:
    Warning: Not converged. Increase D or check for bugs.
  Optional diagnostics (for debugging):
    3. State convergence: $\max_i \|A^{(i)}_{\text{new}} - A^{(i)}_{\text{old}}\|_{\text{Frob}} < \epsilon_{\text{tensor}}$
    4. Truncation stability: $\eta_{\max} < 10^{-7}$ (discarded weight)
end while

Critical Implementation Note — The Local Minimum Problem: Single-site updates with fixed bond dimension D cannot escape local minima during phase transitions or for systems with symmetries. Two mitigation strategies:

1. **Two-Site DMRG**: Use Algorithm 5 for the first 3-5 sweeps to dynamically increase D, then switch to single-site for speed.

2. **Perturbation/Noise Injection**: Before truncation, add random noise to the singular values or density matrix:

$$\rho \to \rho + \alpha \cdot \mathbb{I}, \quad \alpha \sim 10^{-5} \text{ to } 10^{-3} \tag{21}$$

then re-diagonalize. Decrease α linearly with sweep number.

Without these, single-site DMRG may converge to excited states with incorrect energies.

VII. TWO-SITE DMRG (ROBUST AGAINST LOCAL MINIMA)

Algorithm 6 TwoSiteUpdate

Require: Current bond dimensions D_{i-1}, D_i, max bond $D_{\max}$, tolerance ϵ

Contract: $\Theta_{\alpha_{i-1},\sigma_i,\sigma_{i+1},\alpha_{i+1}} = \sum_{\alpha_i} A^{(i)}_{\alpha_{i-1},\sigma_i,\alpha_i} B^{(i+1)}_{\alpha_i,\sigma_{i+1},\alpha_{i+1}}$
Reshape: $\Theta \to$ matrix $(dD_{i-1}) \times (dD_{i+1})$
Build $\mathcal{H}^{(i,i+1)}_{\text{eff}} = \sum_{b_{i-1},b_{i+1}} L_{b_{i-1}} \otimes W^{(i)} \otimes W^{(i+1)} \otimes R_{b_{i+1}}$
Solve for lowest eigenvector $\tilde{\Theta}$ using Lanczos/Davidson
SVD: $\tilde{\Theta} = USV^\dagger$, truncate to $D_{\max}$
$A^{(i)} \leftarrow \text{reshape}(U, (D_{i-1}, d, D))$
$B^{(i+1)} \leftarrow \text{reshape}(SV^\dagger, (D, d, D_{i+1}))$
return Updated tensors, truncation error η

VIII. MPO CONSTRUCTION FOR HAMILTONIANS

A. Nearest-Neighbor Heisenberg Model

Hamiltonian:

$$H = \sum_{i=1}^{L-1} \left[\frac{J}{2}(S_i^+ S_{i+1}^- + S_i^- S_{i+1}^+) + J_z S_i^z S_{i+1}^z - h S_i^z \right] \tag{22}$$

Spin-1/2 Operator Matrices (in basis $\{|\uparrow\rangle, |\downarrow\rangle\}$):

$$S^+ = \begin{pmatrix} 0 & 1 \\ 0 & 0 \end{pmatrix}, \quad S^- = \begin{pmatrix} 0 & 0 \\ 1 & 0 \end{pmatrix}, \quad S^z = \begin{pmatrix} 1/2 & 0 \\ 0 & -1/2 \end{pmatrix}, \quad \mathbb{I} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} \tag{23}$$

MPO Structure: Bond dimension $D_W = 5$. The operator-valued matrix is:

$$\hat{W}^{(i)} = \begin{pmatrix} \hat{\mathbb{I}} & 0 & 0 & 0 & 0 \\ \hat{S}^+ & 0 & 0 & 0 & 0 \\ \hat{S}^- & 0 & 0 & 0 & 0 \\ \hat{S}^z & 0 & 0 & 0 & 0 \\ -h\hat{S}^z & \frac{J}{2}\hat{S}^- & \frac{J}{2}\hat{S}^+ & J_z\hat{S}^z & \hat{\mathbb{I}} \end{pmatrix} \tag{24}$$

Numerical Representation: Each element $\hat{W}^{(i)}_{b,b'}$ is a $d \times d$ matrix (operator). Store as 4-index tensor $W^{(i)}_{b,b',\sigma,\sigma'}$ with shape (D_W, D_W, d, d).

Index Convention: $W^{(i)}_{b_{\text{left}},b_{\text{right}},\sigma_{\text{in}},\sigma_{\text{out}}}$ where:

- $b_{\text{left}}, b_{\text{right}} \in [0, D_W - 1]$: MPO bond indices
- $\sigma_{\text{in}}, \sigma_{\text{out}} \in [0, d-1]$: Physical indices (input/output states)
- Action: $(\hat{W}|\sigma_{\text{in}}\rangle)_{\sigma_{\text{out}}} = W_{\cdot,\cdot,\sigma_{\text{in}},\sigma_{\text{out}}}$

Algorithm 7 ConstructHeisenbergMPO

Require: Chain length L, parameters J, J_z, h, physical dimension $d = 2$
Ensure: MPO tensors $\{W^{(1)}, \ldots, W^{(L)}\}$

Define operator matrices: $S^+, S^-, S^z, \mathbb{I}$ (each $d \times d$)
$D_W \leftarrow 5$ ▷ MPO bond dimension
▷ — Bulk sites $i = 2, \ldots, L-1$ —
for $i = 2$ to $L - 1$ **do**
  Initialize $W^{(i)} \leftarrow \text{zeros}(D_W, D_W, d, d)$ ▷ Shape $(5, 5, 2, 2)$
  $W^{(i)}[0, 0, :, :] \leftarrow \mathbb{I}$ ▷ Row 0, col 0
  $W^{(i)}[1, 0, :, :] \leftarrow S^+$ ▷ Row 1, col 0
  $W^{(i)}[2, 0, :, :] \leftarrow S^-$ ▷ Row 2, col 0
  $W^{(i)}[3, 0, :, :] \leftarrow S^z$ ▷ Row 3, col 0
  $W^{(i)}[4, 0, :, :] \leftarrow -h \cdot S^z$ ▷ Row 4, col 0: local field
  $W^{(i)}[4, 1, :, :] \leftarrow (J/2) \cdot S^-$ ▷ Row 4, col 1
  $W^{(i)}[4, 2, :, :] \leftarrow (J/2) \cdot S^+$ ▷ Row 4, col 2
  $W^{(i)}[4, 3, :, :] \leftarrow J_z \cdot S^z$ ▷ Row 4, col 3
  $W^{(i)}[4, 4, :, :] \leftarrow \mathbb{I}$ ▷ Row 4, col 4
  ▷ All other elements remain zero
end for
▷ — Left boundary site $i = 1$ —
Initialize $W^{(1)} \leftarrow \text{zeros}(1, D_W, d, d)$ ▷ Shape $(1, 5, 2, 2)$
$W^{(1)}[0, 0, :, :] \leftarrow -h \cdot S^z$ ▷ Local field at site 1
$W^{(1)}[0, 1, :, :] \leftarrow (J/2) \cdot S^-$
$W^{(1)}[0, 2, :, :] \leftarrow (J/2) \cdot S^+$
$W^{(1)}[0, 3, :, :] \leftarrow J_z \cdot S^z$
$W^{(1)}[0, 4, :, :] \leftarrow \mathbb{I}$
▷ Equivalently: $W^{(1)}[0, :, :, :] = W^{(\text{bulk})}[4, :, :, :]$ (last row)
▷ — Right boundary site $i = L$ —
Initialize $W^{(L)} \leftarrow \text{zeros}(D_W, 1, d, d)$ ▷ Shape $(5, 1, 2, 2)$
$W^{(L)}[0, 0, :, :] \leftarrow \mathbb{I}$
$W^{(L)}[1, 0, :, :] \leftarrow S^+$
$W^{(L)}[2, 0, :, :] \leftarrow S^-$
$W^{(L)}[3, 0, :, :] \leftarrow S^z$
$W^{(L)}[4, 0, :, :] \leftarrow -h \cdot S^z$ ▷ Local field at site L
▷ Equivalently: $W^{(L)}[:, 0, :, :] = W^{(\text{bulk})}[:, 0, :, :]$ (first column)
return $\{W^{(1)}, W^{(2)}, \ldots, W^{(L)}\}$

Boundary Index Convention:

- $b_{\text{left}} = 4$ (last row of bulk MPO, index 4 in 0-indexed arrays)
- $b_{\text{right}} = 0$ (first column of bulk MPO, index 0)
- Environment initialization: $L^{(1)}_{b_0} = \delta_{b_0,4}$, $R^{(L)}_{b_L} = \delta_{b_L,0}$

Python Implementation Template:

```
import numpy as np

def construct_heisenberg_mpo(L, J=1.0, Jz=1.0, h=0.0):
"""Construct Heisenberg MPO for spin-1/2 chain."""
d = 2  # Physical dimension
D_W = 5  # MPO bond dimension

# Define spin-1/2 operators
Sp = np.array([[0., 1.], [0., 0.]], dtype=complex)
Sm = np.array([[0., 0.], [1., 0.]], dtype=complex)
Sz = np.array([[0.5, 0.], [0., -0.5]], dtype=complex)
Id = np.eye(2, dtype=complex)
```

```
W = []

# Site 1 (left boundary): shape (1, D_W, d, d)
W1 = np.zeros((1, D_W, d, d), dtype=complex)
W1[0, 0] = -h * Sz
W1[0, 1] = (J/2) * Sm
W1[0, 2] = (J/2) * Sp
W1[0, 3] = Jz * Sz
W1[0, 4] = Id
W.append(W1)

# Bulk sites 2 to L-1: shape (D_W, D_W, d, d)
for i in range(2, L):
Wi = np.zeros((D_W, D_W, d, d), dtype=complex)
Wi[0, 0] = Id
Wi[1, 0] = Sp
Wi[2, 0] = Sm
Wi[3, 0] = Sz
Wi[4, 0] = -h * Sz
Wi[4, 1] = (J/2) * Sm
Wi[4, 2] = (J/2) * Sp
Wi[4, 3] = Jz * Sz
Wi[4, 4] = Id
W.append(Wi)

# Site L (right boundary): shape (D_W, 1, d, d)
WL = np.zeros((D_W, 1, d, d), dtype=complex)
WL[0, 0] = Id
WL[1, 0] = Sp
WL[2, 0] = Sm
WL[3, 0] = Sz
WL[4, 0] = -h * Sz
W.append(WL)

return W

# Verification: Check shapes
W_list = construct_heisenberg_mpo(L=10)
assert W_list[0].shape == (1, 5, 2, 2)      # Left boundary
assert W_list[1].shape == (5, 5, 2, 2)      # Bulk
assert W_list[-1].shape == (5, 1, 2, 2)     # Right boundary
```

Verification Tests:

1. **Single bond**: For $L = 2$, contract $W^{(1)}$ and $W^{(2)}$ to get explicit 4×4 Hamiltonian:

$$H_{\text{2-site}} = \sum_b W^{(1)}_{0,b,\sigma_1,\sigma_1'} W^{(2)}_{b,0,\sigma_2,\sigma_2'} \tag{25}$$

   Compare with direct construction of $\frac{J}{2}(S_1^+ S_2^- + S_1^- S_2^+) + J_z S_1^z S_2^z - h(S_1^z + S_2^z)$.

2. **Hermiticity**: For real J, J_z, h, verify $W^{(i)}_{b,b',\sigma,\sigma'} = W^{(i)*}_{b,b',\sigma',\sigma}$ (transpose on physical indices).

3. **Exact diagonalization**: For small $L \leq 10$, compare DMRG energy with exact values in Sec. XI.

Extension to Other Models:

XXZ Model ($J_x = J_y \neq J_z$): Same structure, adjust coefficients.

Long-Range Interactions: Requires larger D_W. For exponentially decaying $\sum_r e^{-r/\xi} S_i^z S_{i+r}^z$, use $D_W \sim \log(L/\xi)$ with auxiliary states.

General Nearest-Neighbor: Any $H = \sum_i h_i + \sum_i H_{i,i+1}$ with h_i (1-body) and $H_{i,i+1}$ (2-body) can be encoded with $D_W = d^2 + 2$ (identity, d^2 operators, Hamiltonian accumulator).

IX. MODEL EXAMPLES

A. Spin-1/2 Heisenberg Chain

$$H = J \sum_{i=1}^{L-1} \vec{S}_i \cdot \vec{S}_{i+1} = J \sum_{i=1}^{L-1} \left[\frac{1}{2} \left(S_i^+ S_{i+1}^- + S_i^- S_{i+1}^+ \right) + S_i^z S_{i+1}^z \right] \tag{26}$$

MPO: Bond dimension $D_W = 5$ as in Sec. VIII.
Numerical Notes: Critical system with logarithmic entanglement growth. Requires $D \sim L^{1/6}$ for fixed accuracy. Extrapolate in η or $1/D$.

B. AKLT Model (Exact MPS Representation)

$$H_{\text{AKLT}} = \sum_{i=1}^{L-1} \left[\vec{S}_i \cdot \vec{S}_{i+1} + \frac{1}{3} (\vec{S}_i \cdot \vec{S}_{i+1})^2 \right] \tag{27}$$

Exact MPS with $D = 2$:

$$A^+ = \begin{pmatrix} 0 & \sqrt{2/3} \\ 0 & 0 \end{pmatrix}, \quad A^0 = \begin{pmatrix} -1/\sqrt{3} & 0 \\ 0 & 1/\sqrt{3} \end{pmatrix}, \quad A^- = \begin{pmatrix} 0 & 0 \\ -\sqrt{2/3} & 0 \end{pmatrix} \tag{28}$$

Verification: $\sum_\sigma A^{\sigma\dagger} A^\sigma = \mathbb{I}_2$, exact energy $E_0 = -(L-1) \cdot 2/3$, $\eta \equiv 0$.
Properties:

- Correlation length $\xi = 1/\ln 3 \approx 0.91$
- String order $\langle S_i^z \exp(i\pi \sum_{k=i+1}^{j-1} S_k^z) S_j^z \rangle = -4/9$
- Open boundaries: two free $S = 1/2$ edge spins

X. CONVERGENCE DIAGNOSTICS

- **Energy**: $\Delta E = |E_n - E_{n-1}| < 10^{-8}$ per site
- **Truncation error**: Monitor $\eta = \sum_{\text{discarded}} s_i^2$, extrapolate $E(\eta) \to 0$
- **Variance**: $\sigma^2 = \langle \psi | H^2 | \psi \rangle - E^2 \sim D^{-2}$ for gapped systems
- **Entanglement entropy**: $S = -\sum s_i^2 \ln s_i^2$ saturation

Computing $\langle H^2 \rangle$: Avoid squaring MPO. Instead: (1) Compute $|\phi\rangle = H|\psi\rangle$, (2) $\langle H^2 \rangle = \langle \phi | \phi \rangle$.

XI. IMPLEMENTATION VERIFICATION CHECKLIST

Phase 0: Index Convention Tests (DO THIS FIRST)
Before implementing any algorithm, verify index conventions:

1. **MPS storage order**:

```
A = np.random.rand(3, 2, 4) + 1j * np.random.rand(3, 2, 4)
assert A.shape == (3, 2, 4)  # (D_left, d, D_right)
assert A[1, 0, 2] == A[1, 0, 2]  # a_left=1, sigma=0, a_right=2
```

2. **MPO physical index order**:

```
Sp = np.array([[0, 1], [0, 0]], dtype=complex)
W = np.zeros((5, 5, 2, 2), dtype=complex)
W[1, 0, :, :] = Sp
# Check: S^+ raises spin, so input=down(0) -> output=up(1)
assert W[1, 0, 0, 1] == 1.0  # sigma_in=0, sigma_out=1
assert W[1, 0, 1, 0] == 0.0  # Cannot lower from up
```

3. **Complex conjugation (no transpose)**:

```
A = np.random.rand(3, 2, 4) + 1j * np.random.rand(3, 2, 4)
A_conj = A.conj()
assert A_conj.shape == A.shape  # Shape unchanged
assert np.allclose(A_conj[1, 0, 2], np.conj(A[1, 0, 2]))
# WRONG: A.conj().T changes shape
```

4. **Reshape for QR**:

```
M = np.random.rand(3, 2, 4)
M_mat = M.reshape(3 * 2, 4, order='C')
# Verify row index mapping: r = a_left * d + sigma
assert np.allclose(M_mat[1 * 2 + 0, 2], M[1, 0, 2])
assert np.allclose(M_mat[2 * 2 + 1, 3], M[2, 1, 3])
```

5. **Left-canonical verification**:

```
A = np.random.rand(3, 2, 4)
A_mat = A.reshape(6, 4, order='C')
Q, R = np.linalg.qr(A_mat)
A_left = Q.reshape(3, 2, 4, order='C')
# Check orthogonality
I = np.einsum('xsa,xsb->ab', A_left.conj(), A_left)
assert np.allclose(I, np.eye(4), atol=1e-14)
```

6. **Environment Hermiticity**:

```
L = np.random.rand(5, 3, 3) + 1j * np.random.rand(5, 3, 3)
# Make Hermitian for each MPO index
for b in range(5):
L[b] = (L[b] + L[b].conj().T) / 2
assert np.allclose(L[b], L[b].conj().T, atol=1e-14)
```

If any Phase 0 test fails, DO NOT proceed. Fix index conventions first.

The following phased testing protocol validates correct implementation of each subsystem before proceeding to full DMRG. Implement in order; failures at early stages indicate index ordering or conjugation bugs that must be fixed before testing variational optimization.

A. Common Failure Modes

If tests fail, check these typical implementation errors:

TABLE II. Phased Implementation Tests

Phase	**Test**	**Expected Result**
0. Index conventions	Run all 6 tests above	All assertions pass. If not, fix storage order before continuing.
1. Canonicalization	QR on random MPS → LeftCanonicalize	Check $\sum_\sigma A^{(i)\sigma\dagger} A^{(i)\sigma} = \mathbb{I}_{D_i}$ for $i < L$ (within 10^{-14}).
2. Truncation	SVD compression of AKLT state ($D_{\max} = 2$)	$\eta = 0$ (exact representation maintained). Bond dimension remains $D = 2$.
3. Environments	Build $L^{(i)}, R^{(i)}$ for AKLT with identity MPO	Environments proportional to identity. Verify $L^{(i)} \propto \delta_{a_{i-1},a'_{i-1}}$ and Hermiticity.
4. MPO construction	Heisenberg $L = 2$, explicit contraction	$E_0 = -0.75$ exactly. Check $H[0,3] = H[3,0] = J/2$ (flip-flop).
5. SingleSite	AKLT model, $L = 10$, $D = 2$, 2 sweeps	Ground state energy $E_0 = -6.0$ (machine precision, $\Delta E < 10^{-12}$).
6. TwoSite	Heisenberg $L = 20$, $D : 4 \to 20$, $\epsilon = 10^{-10}$	Energy extrapolates to -8.772 (Bethe ansatz). Bond dimension adapts dynamically.
7. Convergence	Run single-site DMRG on gapped system	Variance $\sigma^2 < 10^{-6}$ (strictest test). Check $\sigma^2 \sim D^{-2}$ scaling.

- **Phase 0 fails**: Index convention mismatch. Review Section II and verify array access patterns match mathematical notation exactly.
- **Phase 1 fails (non-orthogonal A)**: Reshape index ordering incorrect. Verify row-major (C-order) convention: $r = a_{\text{left}} \cdot d + \sigma$. Check with explicit element access.
- **Phase 2 fails ($\eta > 0$ for AKLT)**: Incorrect SVD truncation logic or singular value sorting (must be descending order). Verify ‘np.argsort(s)[::-1]‘.
- **Phase 3 fails (wrong environments)**:
  - Missing complex conjugate on $A^{(i)*}$ or $B^{(i)*}$ in Eqs. (2)-(3)
  - Used transpose instead of conjugate: ‘A.conj().T‘ instead of ‘A.conj()‘
  - Off-by-one MPO index: $L^{(i)}$ must use $W^{(i-1)}$, not $W^{(i)}$
  - Non-Hermitian environments: check $\|L[b] - L[b]^\dagger\| < 10^{-14}$ for all b
- **Phase 4 fails (wrong AKLT energy or $L = 2$ Heisenberg)**:
  - MPO boundary conditions incorrect: verify $b_{\text{left}} = 4$, $b_{\text{right}} = 0$ for Heisenberg
  - Wrong MPO physical index order: σ (input) vs σ' (output) swapped
  - Environment initialization wrong: $L^{(1)}[4,0,0] \neq 1$ or $R^{(L)}[0,0,0] \neq 1$
- **Phase 5 fails (stuck in local minimum)**:
  - Insufficient bond dimension growth: start with two-site DMRG
  - Missing noise injection: add $\rho \to \rho + \alpha\mathbb{I}$ with $\alpha \sim 10^{-5}$
  - Initial state too far from ground state: use random MPS with small D first
- **Phase 6 fails (variance σ^2 large)**:
  - Lanczos solver tolerance too loose: use ‘tol=1e-10‘ or stricter
  - $\mathcal{H}_{\text{eff}}$ matrix-vector product bug: verify einsum indices in Algorithm 4
  - Not using matrix-free implementation: attempted explicit construction causes numerical errors
- **Memory overflow**: Attempted explicit $\mathcal{H}_{\text{eff}}$ construction. Must use matrix-free Algorithm 4 with ‘scipy.sparse.linalg.Linear

B. Debugging Checklist

When encountering unexpected results, verify in order:

1. **Data types**: All arrays are ‘complex128‘, not ‘float64‘ or ‘complex64‘
2. **Shapes**: Print and verify tensor shapes at each step

```
print(f"A.shape = {A.shape}, expected ({D_left}, {d}, {D_right})")
print(f"L.shape = {L.shape}, expected ({D_W}, {D_left}, {D_left})")
```

3. **Orthogonality**: After canonicalization, check $A^\dagger A = I$ or $BB^\dagger = I$
4. **Hermiticity**: Environments must be Hermitian for each MPO index
5. **Normalization**: MPS norm $\langle\psi|\psi\rangle = 1$ within 10^{-12}
6. **Energy bounds**: Ground state energy must be below any excited state
7. **Variance**: For converged state, $\sigma^2 = \langle H^2\rangle - E^2 < 10^{-6}$
8. **Truncation error**: Sum of discarded singular values squared: $\eta = \sum_{i>D} s_i^2$

C. Reference Values for Heisenberg Chain

For $S = 1/2$ Heisenberg with open boundaries ($J = 1$, $J_z = 1$, $h = 0$), exact diagonalization:

L	E_0/J	E_0/L
2	-0.750000000000	-0.375
4	-1.616025403784	-0.404
6	-2.493577131515	-0.416
8	-3.374932109083	-0.422
10	-4.258035965	-0.426
20	-8.772 (Bethe)	-0.439
∞	$-0.4431471805599453 \times L$	-0.443147

Thermodynamic limit: $e_0 = E_0/L \to -0.4431471805599453$ (Bethe ansatz, 16-digit precision).
MPO Verification Procedure:

1. Construct MPO for $L = 2$ using Algorithm 6
2. Contract to explicit 4×4 Hamiltonian:

```
# Contract MPO to get explicit Hamiltonian
H_explicit = np.zeros((4, 4), dtype=complex)
for b in range(D_W):
H_explicit += np.kron(W[0, b, :, :], W[b, 0, :, :])
# Basis order: |00>, |01>, |10>, |11> (up-up, up-down, down-up, down-down)
```

3. Diagonalize and verify lowest eigenvalue $= -0.75$:

```
eigvals, eigvecs = np.linalg.eigh(H_explicit)
E0 = eigvals[0]
assert np.abs(E0 - (-0.75)) < 1e-12, f"Wrong energy: {E0}"
```

4. Check matrix elements match analytical form:

$$H[0,3] = H[3,0] = J/2 = 0.5 \quad \text{(flip-flop: } S^+S^- \text{ and } S^-S^+\text{)} \tag{29}$$
$$H[1,1] = H[2,2] = -J_z/4 = -0.25 \quad \text{(diagonal: } S^zS^z \text{ for opposite spins)} \tag{30}$$
$$H[0,0] = J_z/4 - h = 0.25 \quad \text{(both up)} \tag{31}$$
$$H[3,3] = J_z/4 + h = 0.25 \quad \text{(both down)} \tag{32}$$

```
assert np.abs(H_explicit[0, 3] - 0.5) < 1e-14
assert np.abs(H_explicit[3, 0] - 0.5) < 1e-14
assert np.abs(H_explicit[1, 1] - (-0.25)) < 1e-14
assert np.abs(H_explicit[2, 2] - (-0.25)) < 1e-14
```

5. Verify Hermiticity:

```
assert np.allclose(H_explicit, H_explicit.conj().T, atol=1e-14)
```

Use these tests to verify MPO construction before attempting DMRG on larger systems.

D. Performance Benchmarks

Expected computational costs for single-site DMRG sweep (one left + one right):

L	D	Time/sweep	Memory
10	20	∼0.1 s	∼1 MB
50	50	∼2 s	∼10 MB
100	100	∼20 s	∼100 MB
100	200	∼2 min	∼500 MB

Hardware: Single CPU core, NumPy with OpenBLAS/MKL. GPU acceleration possible but requires custom kernels for einsum operations.

Scaling:

- Time per sweep: $\mathcal{O}(LD^3d^2D_W N_{\text{Lanczos}})$ where $N_{\text{Lanczos}} \sim$ 10-20
- Memory: $\mathcal{O}(LD^2d + D^3dD_W)$ (dominated by environments and MPS storage)
- Two-site DMRG: ∼3-5× slower than single-site due to larger effective Hamiltonian

If your implementation is significantly slower, check:

- Using optimized BLAS (not default NumPy): `np.show_config()`
- Einsum optimization: `np.einsum(..., optimize=True)` or use `opt_einsum` package
- Avoiding unnecessary copies: use in-place operations where possible
- Not constructing explicit $\mathcal{H}_{\text{eff}}$ matrix

[1] U. Schollwöck, *The density-matrix renormalization group in the age of matrix product states*, Annals of Physics **326**, 96–192 (2011).